```
clear; clc; close all;
run(fullfile(fileparts(fileparts(mfilename('fullpath'))),
'setup_project.m'));
m = 4;
strats = {@choose_leaders, @base_random, @base_degree, @base_closest};
names  = {'Choose Leaders', 'Random', 'Degree', 'Closest'};
n_s = numel(strats);
seed_sets = {1:30, 1001:1030};
set_names = {'Tuning', 'Held-out'};
best = zeros(size(seed_sets));
median_seeds = zeros(size(seed_sets));
worst = zeros(size(seed_sets));
for q = 1:2
    seeds = seed_sets{q};
    J = zeros(numel(seeds), n_s);
    for s = 1:numel(seeds)
        D = make_deployment(seeds(s));
        for k = 1:n_s
            S = get_leaders(strats{k}, D, m, seeds(s));
            J(s,k) = score_mission(D, S);
        end
    end
    figure;
    boxplot(J, 'Labels', names)
    plot_title = sprintf('Percent reduction in mean squared distance to g
for each leader strategy (%s)', set_names{q});
    title(plot_title);
    [~, bestIdx] = max(J(:,1));
    fprintf('Best seed for choose_leaders:   %d, J = %.3f\n',
seeds(bestIdx), J(bestIdx,1));
    medJ = median(J(:,1));
    [~, medIdx] = min(abs(J(:,1) - medJ));
    fprintf('Median seed for choose_leaders: %d, J = %.3f (median J = %.3f)
\n', ...
    seeds(medIdx), J(medIdx,1), medJ);
    [~, worstIdx] = min(J(:,1));
    fprintf('Worst seed for choose_leaders:  %d, J = %.3f\n',
seeds(worstIdx), J(worstIdx,1));
    results{q} = J;
    best(q) = seeds(bestIdx);
    median_seeds(q) = seeds(medIdx);
    worst(q) = seeds(worstIdx);
end
seeds_export=[best, median_seeds, worst];
seeds = 2001:2050;
t = zeros(size(seeds));
D = make_deployment(seeds(1));
choose_leaders(D.G, D.P0, D.faulty, D.g, m);
for s = 1:numel(seeds)
    D = make_deployment(seeds(s));
    tic;
    choose_leaders(D.G, D.P0, D.faulty, D.g, m);
```

```
    t(s) = toc;
end
fprintf('runtime: mean %.3f s, max %.3f s\n', mean(t), max(t));

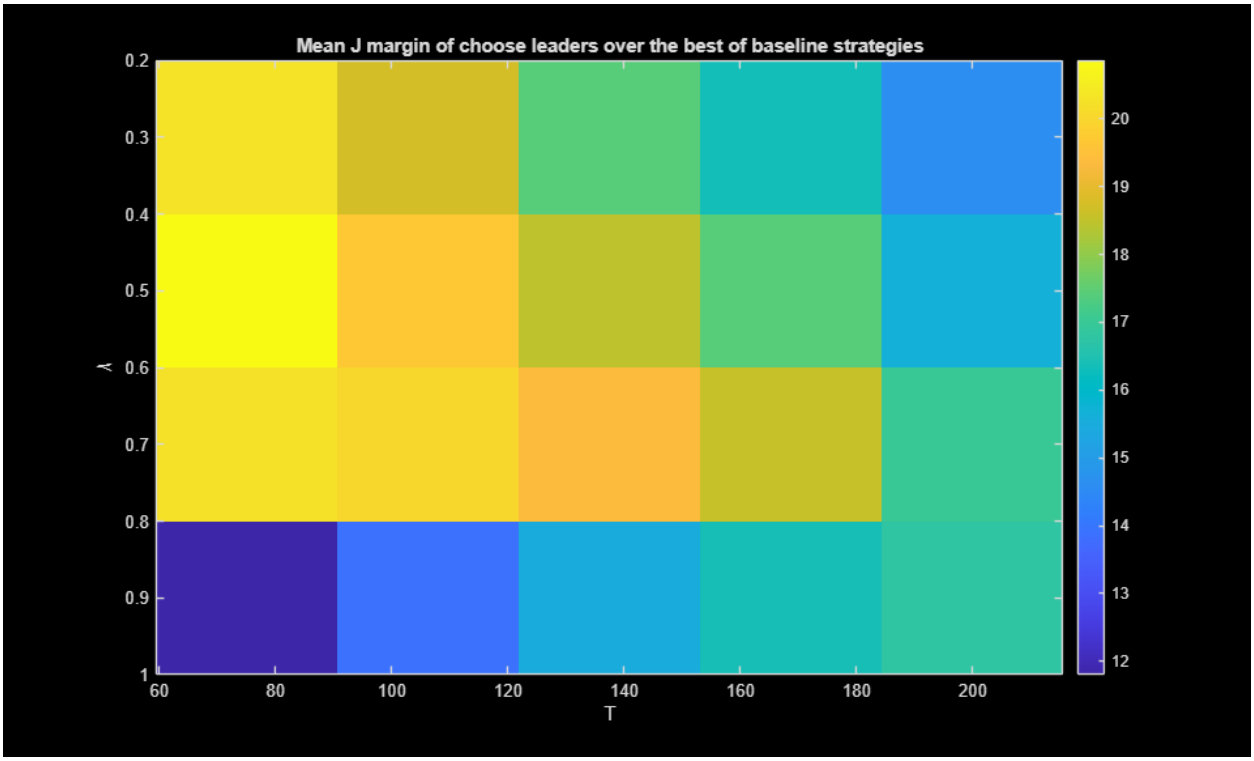
lams = [0.3 0.5 0.7 0.9];   ts = [75 100 125 150 200];
seeds = 1001:1030;
n_seed = numel(seeds);
n_s = numel(strats);
n_lam  = numel(lams);
n_t = numel(ts);
J = nan(n_seed, n_s, n_lam, n_t);
% J(seed, strategy, lambda index, T index)
for s = 1:n_seed
    D = make_deployment(seeds(s));
    for k = 1:n_s
        S = get_leaders(strats{k}, D, m, seeds(s));
        for a = 1:n_lam
            for b = 1:n_t
                J(s, k, a, b) = score_mission(D, S, lams(a), ts(b));
            end
        end
    end
end
choose_leaders_mean = zeros(n_lam, n_t);
best_base = zeros(n_lam, n_t);
for a = 1:n_lam
    for b = 1:n_t
        mine  = J(:, 1, a, b);
        bases = J(:, 2:4, a, b);
        choose_leaders_mean(a, b) = mean(mine);
        base_means = mean(bases);
        best_base(a, b) = max(base_means);
    end
end
margin = choose_leaders_mean - best_base;
figure; imagesc(ts, lams, margin);
colorbar;
xlabel('T'); ylabel('\lambda'); title('Mean J margin of choose leaders over
the best of baseline strategies');
m = 4;
for q = 1:numel(seeds_export)
    sd = seeds_export(q);
    D  = make_deployment(sd);
    rng(sd);
    L(1).name = 'Mine';     L(1).S = choose_leaders(D.G, D.P0, D.faulty,
D.g, m);
    L(2).name = 'Degree';   L(2).S = base_degree(D.G, D.P0, D.faulty, D.g,
m);
    L(3).name = 'Closest';  L(3).S = base_closest(D.G, D.P0, D.faulty, D.g,
m);
    L(4).name = 'Random';   L(4).S = base_random(D.G, D.P0, D.faulty, D.g,
m);
    export_for_viewer(D, L, sprintf('seed%d.json', sd));
    fprintf('Seed %d\n', sd);
```

```
    for k = 1:numel(L)
        fprintf('  %-8s J = %6.2f  (T=150)   %6.2f  (T=125)\n', L(k).name,
...
            score_mission(D, L(k).S, 0.5, 150), score_mission(D, L(k).S,
0.5, 125));
    end
end

function S = get_leaders(f, D, m, seed)
    rng(seed);
    S = f(D.G, D.P0, D.faulty, D.g, m);
end
function S = base_random(G, P0, faulty, g, m)
    w = setdiff((1:size(G,1))', faulty(:));
    S = w(randperm(numel(w), m));
end
function S = base_degree(G, P0, faulty, g, m)
    d = G * ones(size(G,1), 1);   d(faulty) = -Inf;
    [~, i] = sort(d, 'descend');  S = i(1:m);
end
function S = base_closest(G, P0, faulty, g, m)
    d = sum((P0 - g).^2, 2);      d(faulty) = Inf;
    [~, i] = sort(d, 'ascend');   S = i(1:m);
end
```

```
Best seed for choose_leaders:   24, J = 88.162
Median seed for choose_leaders: 19, J = 73.717 (median J = 73.814)
Worst seed for choose_leaders:  25, J = 51.141
Best seed for choose_leaders:   1029, J = 89.139
Median seed for choose_leaders: 1027, J = 67.997 (median J = 67.983)
Worst seed for choose_leaders:  1011, J = 49.058
runtime: mean 0.000 s, max 0.002 s
Saved seed24.json. Load it in the Swarm Rally Race viewer.
Seed 24
  Mine     J =  89.96  (T=150)    88.16  (T=125)
  Degree   J =  81.23  (T=150)    78.57  (T=125)
  Closest  J =   0.19  (T=150)     0.19  (T=125)
  Random   J =  81.08  (T=150)    77.62  (T=125)
Saved seed1029.json. Load it in the Swarm Rally Race viewer.
Seed 1029
  Mine     J =  90.92  (T=150)    89.14  (T=125)
  Degree   J =  90.94  (T=150)    89.17  (T=125)
  Closest  J =   0.20  (T=150)     0.18  (T=125)
  Random   J =  77.71  (T=150)    73.63  (T=125)
Saved seed19.json. Load it in the Swarm Rally Race viewer.
Seed 19
  Mine     J =  77.37  (T=150)    73.72  (T=125)
  Degree   J =  72.02  (T=150)    69.07  (T=125)
  Closest  J =   4.25  (T=150)     3.97  (T=125)
  Random   J =  49.89  (T=150)    45.52  (T=125)
Saved seed1027.json. Load it in the Swarm Rally Race viewer.
Seed 1027
  Mine     J =  68.07  (T=150)    68.00  (T=125)
  Degree   J =  21.05  (T=150)    19.60  (T=125)
```

```
  Closest  J =  15.77  (T=150)    14.45  (T=125)
  Random   J =  71.34  (T=150)    68.36  (T=125)
Saved seed25.json. Load it in the Swarm Rally Race viewer.
Seed 25
  Mine     J =  52.94  (T=150)    51.14  (T=125)
  Degree   J =  52.10  (T=150)    51.29  (T=125)
  Closest  J =  25.04  (T=150)    25.62  (T=125)
  Random   J =  57.25  (T=150)    55.43  (T=125)
Saved seed1011.json. Load it in the Swarm Rally Race viewer.
Seed 1011
  Mine     J =  50.19  (T=150)    49.06  (T=125)
  Degree   J =  20.92  (T=150)    19.91  (T=125)
  Closest  J =  18.32  (T=150)    17.06  (T=125)
  Random   J =  31.33  (T=150)    30.78  (T=125)
```



Published with MATLAB® R2026a